

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Phát triển Ứng dụng
Internet of Things - CO3037

Đề tài
HỆ THỐNG GIÁM SÁT VƯỜN THÔNG MINH

Giảng viên hướng dẫn: Lê Trọng Nhân
Lớp L01

STT	Họ tên SV	MSSV	Tên lớp
1	Hoàng Ngô Thiên Phúc	2212612	L01
2	Phạm Anh Hùng	2211346	L01
3	Nguyễn Thế Mạnh	2211990	L01

TP. HỒ CHÍ MINH, THÁNG 04/2025



Mục lục

1	Giới Thiệu	4
1.1	Mục Tiêu Cụ Thể	5
2	Thiết Bị Sử Dụng	5
2.1	Thiết Bị Đầu Vào	5
2.2	Thiết Bị Đầu Ra	6
3	Kiến Trúc Hệ Thống	7
3.1	Cấu Trúc Thư Mục	7
3.2	Sơ Đồ Luồng Dữ Liệu	8
3.3	Danh Sách RTOS Tasks	9
4	Kiến Trúc Phần Mềm Của Hệ Thống	9
4.1	Mô Hình Phân Lớp	9
4.2	Quy Trình Khởi Động (Boot Sequence)	10
4.3	Nguyên Tắc Module Hóa	10
5	Các Giải Pháp Triển Khai	10
5.1	Task 1: LED Blink Theo Nhiệt Độ	10
5.2	Task 2: NeoPixel RGB Theo Độ Ẩm	11
5.3	Task 3: LCD và Loại Bỏ Biến Toàn Cục	11
5.3.1	Struct SensorData Thay Thế Biến Toàn Cục	12
5.3.2	Mailbox Queue	12
5.3.3	Logic Trạng Thái LCD	12
5.4	Task 4: Web Server AP Mode	12
5.4.1	Kiến Trúc WebSocket	13
5.4.2	Chế Độ WiFi AP+STA Đồng Thời	13
5.4.3	Dashboard 4 Trang	13
5.5	Task 5: TinyML — Phát Hiện Bất Thường	17
5.5.1	Dataset	17
5.5.2	Kiến Trúc Mô Hình ANN	18
5.5.3	Tiền Xử Lý Z-score	18
5.5.4	Kết Quả Đánh Giá	18
5.6	Task 6: CoreIOT Cloud (MQTT)	18
5.6.1	Cấu Hình MQTT	18



5.6.2	Đồng Bộ Với WiFi	19
5.6.3	Payload Telemetry	19
6	Cơ Chế Đồng Bộ (Semaphore & Queue)	19
6.1	Tổng Quan Các Đối Tượng RTOS	19
6.2	Pattern: Edge Detection	20
6.3	Pattern: Mailbox Queue với Overwrite	20
7	ThingsBoard Rule Chain	21
7.1	Luồng Dữ Liệu	21
7.2	Node Script JavaScript	21
7.3	Gán Rule Chain Vào Device Profile	22
7.4	Lợi Ích	23
8	Mã Nguồn — Các Đoạn Code Quan Trọng	23
8.1	Khởi Tạo Hệ Thống	23
8.2	Edge Detection Semaphore (Task 1)	24
8.3	Mailbox Queue (Task 3)	25
8.4	TFLite Micro Inference (Task 5)	25
8.5	MQTT CoreIOT (Task 6)	25
9	Kết Luận	26
9.1	Tóm Tắt Thành Tựu	26
9.2	Bài Học Kinh Nghiệm	27
	Mã Nguồn	27
	Tài Liệu Tham Khảo	28

1 Giới Thiệu

Dự án xây dựng một hệ thống IoT nhúng toàn diện trên bo mạch **YOLO UNO** (chip ESP32-S3 Dual Core 240 MHz, 4 MB Flash, 8 MB PSRAM) sử dụng framework Arduino trên PlatformIO kết hợp với **FreeRTOS** để quản lý đa nhiệm theo thời gian thực.

Hệ thống thu thập dữ liệu từ cảm biến nhiệt độ/độ ẩm DHT20, xử lý và phân phối qua các cơ chế RTOS (Queue và Semaphore), điều khiển LED, NeoPixel, LCD, đồng thời cung cấp giao diện web, chạy mô hình TinyML phát hiện bất thường và đẩy dữ liệu lên nền tảng đám mây CoreIOT.

Những điểm nổi bật của hệ thống:

- **Đa nhiệm theo thời gian thực:** 8 FreeRTOS Task chạy song song, mỗi task đảm nhận một chức năng độc lập.
- **Truyền dữ liệu an toàn:** Dữ liệu cảm biến truyền hoàn toàn qua Queue và Semaphore — không sử dụng biến toàn cục.
- **Giao diện web thời gian thực:** Dashboard 4 trang với WebSocket, đồng hồ gauge và điều khiển LED, NeoPixel (bật/tắt) cùng relay động.
- **TinyML trên vi điều khiển:** Mô hình ANN 33 tham số (1928 bytes) chạy trực tiếp trên ESP32-S3 để phát hiện bất thường môi trường.
- **Kết nối đám mây:** Đẩy telemetry lên CoreIOT qua giao thức MQTT theo chuẩn ThingsBoard.

1.1 Mục Tiêu Cụ Thể

Task	Nhiệm vụ
Task 1	LED đơn chớp theo ít nhất 3 điều kiện nhiệt độ; dùng semaphore đồng bộ
Task 2	Điều khiển NeoPixel với ít nhất 3 mức màu theo độ ẩm; dùng semaphore
Task 3	Giám sát T/H hiển thị LCD với ít nhất 3 trạng thái; loại bỏ biến toàn cục
Task 4	Web server chế độ AP; giao diện điều khiển ít nhất 2 thiết bị, 2 nút
Task 5	Triển khai TinyML trên vi điều khiển; mô tả dataset; đánh giá độ chính xác
Task 6	Publish sensor data lên CoreIOT (chế độ STA); dùng token xác thực

Bảng 1.1: Danh sách các task theo yêu cầu đề bài

2 Thiết Bị Sử Dụng

2.1 Thiết Bị Đầu Vào

1. Cảm biến DHT20

- **Chức năng:** Đo nhiệt độ và độ ẩm không khí.
- **Giao tiếp:** I2C, địa chỉ 0x38, SDA = GPIO 11, SCL = GPIO 12.
- **Chu kỳ đọc:** 5 giây/lần trong task `temp_humi_monitor`.
- **Thư viện:** AHTxx.

2. Nút nhấn BOOT (GPIO 0)

- **Chức năng:** Xóa toàn bộ cấu hình WiFi và CoreIOT lưu trong LittleFS khi giữ nút quá 2 giây.
- **Cơ chế:** Task `Task_Toogle_BOOT` đếm thời gian giữ nút; khi vượt ngưỡng sẽ gọi `LittleFS.format()` và khởi động lại thiết bị.

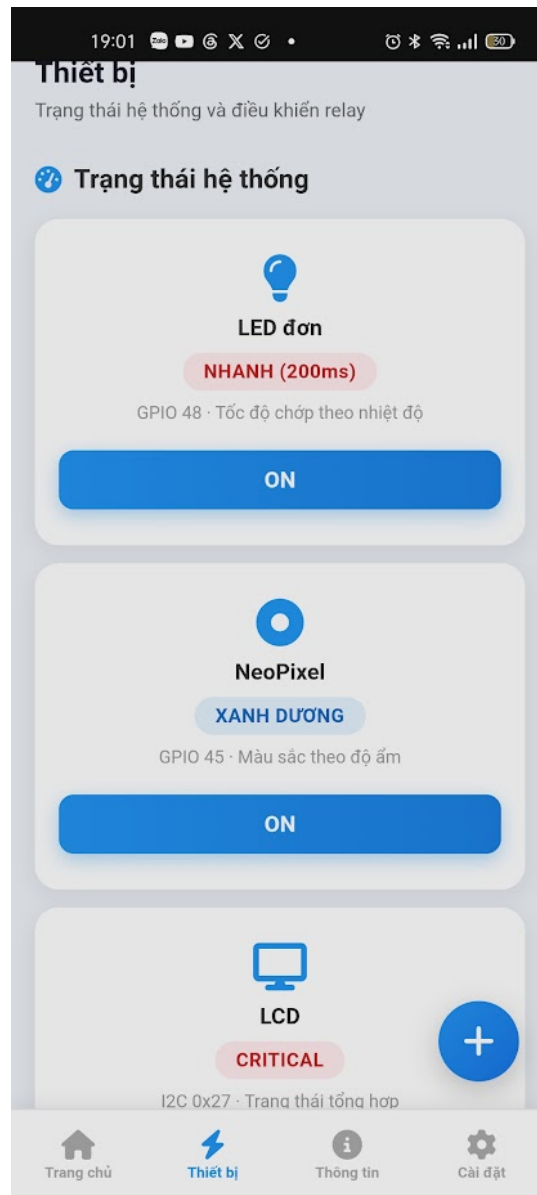
2.2 Thiết Bị Đầu Ra

1. Thiết bị điều khiển và chấp hành

- **LED đơn (GPIO 48):** Chớp với 3 tốc độ tương ứng 3 mức nhiệt độ — 200 ms (nóng), 1000 ms (bình thường), 2000 ms (lạnh). Được bật/tắt qua nút toggle trên trang Device của dashboard; khi tắt, LED giữ mức LOW.
- **NeoPixel RGB (GPIO 45):** LED addressable WS2812B, thư viện `Adafruit_NeoPixel`, độ sáng 50%. Đổi màu theo 3 mức độ ẩm: Đỏ ($H < 40\%$), Xanh lá ($40\% \leq H \leq 60\%$), Xanh dương ($H > 60\%$). Được bật/tắt qua nút toggle trên dashboard; khi tắt, NeoPixel tắt hẳn (`pixels.clear()`) nhưng vẫn theo dõi màu hiện tại để khôi phục ngay khi bật lại.
- **Relay động:** Người dùng thêm relay qua Web dashboard bằng cách nhập tên và số GPIO; firmware thực hiện `digitalWrite(gpio, state)` theo lệnh WebSocket.

2. Thiết bị hiển thị

- **Màn hình LCD 16x2 I2C (địa chỉ 0x27):** Hiển thị trạng thái hệ thống (NORMAL / WARNING / CRITICAL) và giá trị T, H cập nhật mỗi 500 ms.



Hình 2.1: Bo mạch YOLO UNO và các thiết bị ngoại vi kết nối

3 Kiến Trúc Hệ Thống

3.1 Cấu Trúc Thư Mục

Mã nguồn được tổ chức theo hướng module hóa, mỗi thư mục con tương ứng một nhóm chức năng:

- `src/actuators/` — Task 1 (LED), Task 2 (NeoPixel)

- `src/sensors/` — Task 3 (DHT20 monitor, LCD task)
- `src/connectivity/` — Quản lý WiFi, LittleFS, nút BOOT
- `src/web_services/` — Task 4 (AsyncWebServer + WebSocket handler)
- `src/cloud/` — Task 6 (MQTT CoreIOT)
- `src/tinym1/` — Task 5 (TFLite Micro inference, model header, dataset)
- `src/global.cpp` + `include/global.h` — Khởi tạo và khai báo Queue, Semaphore, struct `SensorData`
- `data/` — File web tĩnh phục vụ qua LittleFS (HTML, JS, CSS)

3.2 Sơ Đồ Luồng Dữ Liệu

Toàn bộ dữ liệu cảm biến được phân phối qua hai cơ chế RTOS:

- **xSensorQueue (Mailbox, depth=1):** Task `temp_humi_monitor` ghi struct `SensorData{temp, humidity, lcd_status}` vào Queue bằng `xQueueOverwrite`. Các task LCD, WebServer và TinyML đọc bằng `xQueuePeek` (không xóa item).
- **9 Binary Semaphore:** 3 semaphore nhiệt độ (Low/Normal/High), 3 semaphore độ ẩm (Low/Normal/High), 3 semaphore trạng thái (Normal/Warning/Critical) — được give theo pattern *edge detection* khi phát hiện thay đổi mức.

3.3 Danh Sách RTOS Tasks

Task	Stack	Ưu tiên	Vai trò
LED Blink	2048 B	2	Điều khiển LED GPIO 48 theo semaphore nhiệt độ
NEO Blink	2048 B	2	Điều khiển NeoPixel GPIO 45 theo semaphore độ ẩm
TEMP HUMI	4096 B	2	Đọc DHT20, gửi Queue + give Semaphore trạng thái
LCD Display	4096 B	2	Nhận Queue, hiển thị LCD 16×2 I2C
TinyML	4096 B	2	TFLite Micro inference, phát hiện bất thường
CoreIOT	8192 B	2	MQTT publish telemetry lên <code>app.coreiot.io</code>
WebServer	8192 B	2	AsyncWebServer + WebSocket real-time dashboard
BOOT Toggle	4096 B	2	Giám sát GPIO 0, xóa LittleFS khi giữ >2 s

Bảng 3.1: Danh sách RTOS Task trong hệ thống (hàm tương ứng xem `src/main.cpp`)

4 Kiến Trúc Phần Mềm Của Hệ Thống

4.1 Mô Hình Phân Lớp

Phần mềm được tổ chức theo 5 lớp từ dưới lên:

1. **Hardware:** ESP32-S3 240 MHz Dual Core.
2. **HAL (FreeRTOS + Arduino):** Scheduler, GPIO, I2C, WiFi stack.
3. **Connectivity:** `task_wifi` (AP+STA), `task_check_info` (LittleFS).
4. **Data Bus:** `xSensorQueue` (Mailbox) + 9 Binary Semaphore.
5. **Application:** 8 FreeRTOS Task xử lý nghiệp vụ cụ thể.

4.2 Quy Trình Khởi Động (Boot Sequence)

1. `setup()` gọi `init_globals()` — tạo toàn bộ Queue và Semaphore **trước** khi đăng ký bất kỳ task nào.
2. Lần lượt gọi `xTaskCreate()` cho 8 task.
3. FreeRTOS Scheduler bắt đầu điều phối các task.

Quy tắc quan trọng: `init_globals()` bắt buộc phải chạy trước `xTaskCreate()`. Nếu Queue/Semaphore chưa tạo mà task đã gọi `xSemaphoreTake()`, firmware sẽ crash do NULL pointer dereference.

4.3 Nguyên Tắc Module Hóa

- **Single Responsibility:** Mỗi file `.cpp` chứa đúng 1 FreeRTOS task với 1 chức năng.
- **Không biến toàn cục sensor:** Dữ liệu truyền qua `xSensorQueue`; trạng thái thiết bị qua Semaphore. Ngoại lệ có kiểm soát: hai flag `led_blinky_enabled` và `neo_blinky_enabled` là biến toàn cục kiểu `bool` dùng để bật/tắt actuator từ WebSocket — không mang dữ liệu sensor.
- **Producer-Consumer rõ ràng:** `temp_humi_monitor` là producer duy nhất; các task còn lại là consumer thuần túy.
- **Cấu hình tập trung:** Ngưỡng T/H định nghĩa trong `include/global.h` — thay đổi một chỗ ảnh hưởng toàn hệ thống.

5 Các Giải Pháp Triển Khai

5.1 Task 1: LED Blink Theo Nhiệt Độ

Yêu cầu: LED chớp với 3 tốc độ khác nhau tương ứng ít nhất 3 mức nhiệt độ, sử dụng Semaphore để đồng bộ.

Ngưỡng (định nghĩa trong `include/global.h`): `TEMP_NORMAL_LOW = 25.0°C`, `TEMP_NORMAL_HIGH = 30.0°C`.

Giải pháp — Edge Detection Semaphore: Task `temp_humi_monitor` theo dõi biến `last_temp` và chỉ `xSemaphoreGive` khi phát hiện thay đổi mức, không give ở mỗi chu kỳ đọc. Task `led_blinky` poll 3 semaphore với `timeout = 0` (non-blocking) và duy trì `blinkInterval` cho đến khi có semaphore mới.

Mức nhiệt độ	Semaphore	Khoảng chớp
Lạnh: $T < 25C$	xTempLowSemaphore	2000 ms
Bình thường: $25 \leq T \leq 30C$	xTempNormalSemaphore	1000 ms
Nóng: $T > 30C$	xTempHighSemaphore	200 ms

Bảng 5.1: Ánh xạ mức nhiệt độ sang tốc độ LED

Điều Khiển ON/OFF qua Dashboard (Task 4): Flag toàn cục `led_blinky_enabled` (mặc định `true`) nhận lệnh từ nút toggle trên trang Device. Khi `false`, task giữ LED ở mức LOW và `vTaskDelay(500 ms)` thay vì thực hiện chu kỳ chớp.

5.2 Task 2: NeoPixel RGB Theo Độ Ẩm

Yêu cầu: NeoPixel hiển thị 3 màu tương ứng 3 mức độ ẩm.

Ngưỡng: `HUMI_NORMAL_LOW = 40.0%`, `HUMI_NORMAL_HIGH = 60.0%`.

Mức độ ẩm	Semaphore	Màu	RGB
Khô: $H < 40\%$	xHumiLowSemaphore	Đỏ	(255, 0, 0)
Bình thường: $40 \leq H \leq 60\%$	xHumiNormalSemaphore	Xanh lá	(0, 255, 0)
Ẩm: $H > 60\%$	xHumiHighSemaphore	Xanh dương	(0, 0, 255)

Bảng 5.2: Ánh xạ mức độ ẩm sang màu NeoPixel

Cơ chế edge detection và poll semaphore tương tự Task 1. NeoPixel khởi tạo với độ sáng 50% bằng `pixels.setBrightness(50)`, thư viện `Adafruit_NeoPixel`.

Điều Khiển ON/OFF qua Dashboard (Task 4): Flag `neo_blinky_enabled` nhận lệnh từ nút toggle trên trang Device. Task tách biệt hai pha: (1) luôn cập nhật `current_color` từ semaphore kể cả khi disabled; (2) chỉ gọi `pixels.show()` khi màu hoặc trạng thái bật/tắt thay đổi. Khi bật lại, màu cũ được khôi phục ngay lập tức mà không cần chờ thay đổi semaphore mới.

5.3 Task 3: LCD và Loại Bỏ Biến Toàn Cục

Yêu cầu: Hiển thị 3 trạng thái trên LCD, loại bỏ **hoàn toàn** biến toàn cục liên quan đến cảm biến.

5.3.1 Struct SensorData Thay Thế Biến Toàn Cục

Thay vì dùng `float glob_temp; float glob_humi;`, toàn bộ dữ liệu sensor được đóng gói trong struct duy nhất:

Listing 5.1: Struct SensorData trong `include/global.h`

```
1 struct SensorData {  
2     float temperature;  
3     float humidity;  
4     int    lcd_status;    // 0=NORMAL, 1=WARNING, 2=CRITICAL  
5 };
```

5.3.2 Mailbox Queue

Queue được tạo với `depth = 1` (mailbox pattern). Producer dùng `xQueueOverwrite` (không block), consumer dùng `xQueuePeek` (không xóa item):

- LCD, WebServer và TinyML cùng đọc một item — không task nào "ăn mất" dữ liệu của task khác.
- Producer không bao giờ bị block khi queue đầy.

5.3.3 Logic Trạng Thái LCD

- **CRITICAL (2):** $T > 30C$
- **WARNING (1):** $T \geq 25C$ hoặc $H < 40\%$ hoặc $H > 60\%$
- **NORMAL (0):** Tất cả điều kiện còn lại

5.4 Task 4: Web Server AP Mode

Yêu cầu: Web server hoạt động ở chế độ AP, điều khiển ít nhất 2 thiết bị, giao diện có ít nhất 2 nút.

Hai thiết bị được điều khiển: LED đơn (GPIO 48) và NeoPixel (GPIO 45) — mỗi thiết bị có một nút ON/OFF toggle riêng trên trang Device của dashboard.

5.4.1 Kiến Trúc WebSocket

Sử dụng **ESPAsyncWebServer + AsyncWebSocket** thay vì HTTP polling, cho phép cập nhật dữ liệu thời gian thực không cần tải lại trang. Endpoint: `/ws`, HTTP port 80, file tĩnh từ LittleFS.

WebSocket handler phân loại theo trường `page` trong JSON message:

page	Trường bổ sung	Hành động
actuator	device, status	Bật/tắt LED (<code>led_blinky_enabled</code>) hoặc NeoPixel (<code>neo_blinky_enabled</code>)
device	value.gpio, value.status	<code>digitalWrite</code> relay tùy chỉnh theo GPIO
scan_wifi	—	Quét danh sách WiFi
setting	value.ssid, value.password, ...	Lưu cấu hình, reboot

Bảng 5.3: Phân loại WebSocket message theo trường `page`

Khi người dùng nhấn nút toggle, JS gửi message `{page:"actuator", device:"led"/"neo", status:"ON"/"OFF"}`. Firmware cập nhật flag toàn cục tức thì; trạng thái được đồng bộ về client qua trường `led_en/neo_en` trong mỗi payload sensor broadcast.

5.4.2 Chế Độ WiFi AP+STA Đồng Thời

Listing 5.2: Cấu hình WIFI_AP_STA trong `task_wifi.cpp`

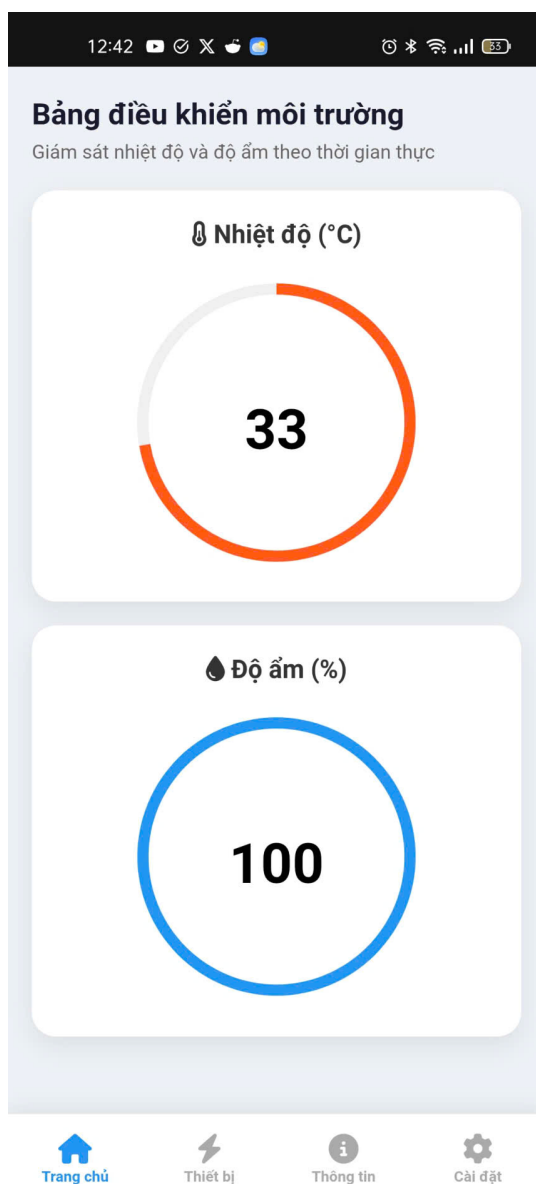
```
1 WiFi.mode(WIFI_AP_STA);
2 WiFi.softAP("ESP32 LOCAL", "12345678"); // IP: 192.168.4.1
```

Đảm bảo dashboard luôn truy cập được qua IP 192.168.4.1 ngay cả khi chưa có kết nối Internet.

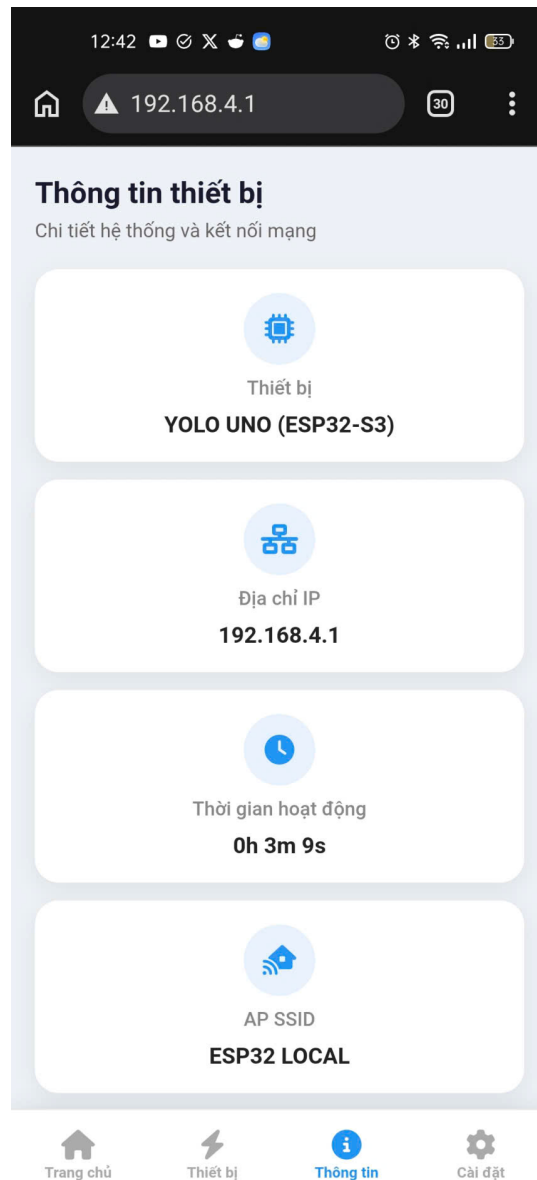
5.4.3 Dashboard 4 Trang

1. **Home:** Hai đồng hồ gauge (JustGage + Raphael.js) hiển thị T/H thời gian thực.
2. **Device:** Hiển thị trạng thái LCD (read-only); nút ON/OFF toggle cho **LED đơn** và **NeoPixel** — đây là 2 thiết bị và 2 nút điều khiển theo yêu cầu Task 4; nút thêm/xóa **relay động** theo GPIO tùy chỉnh.

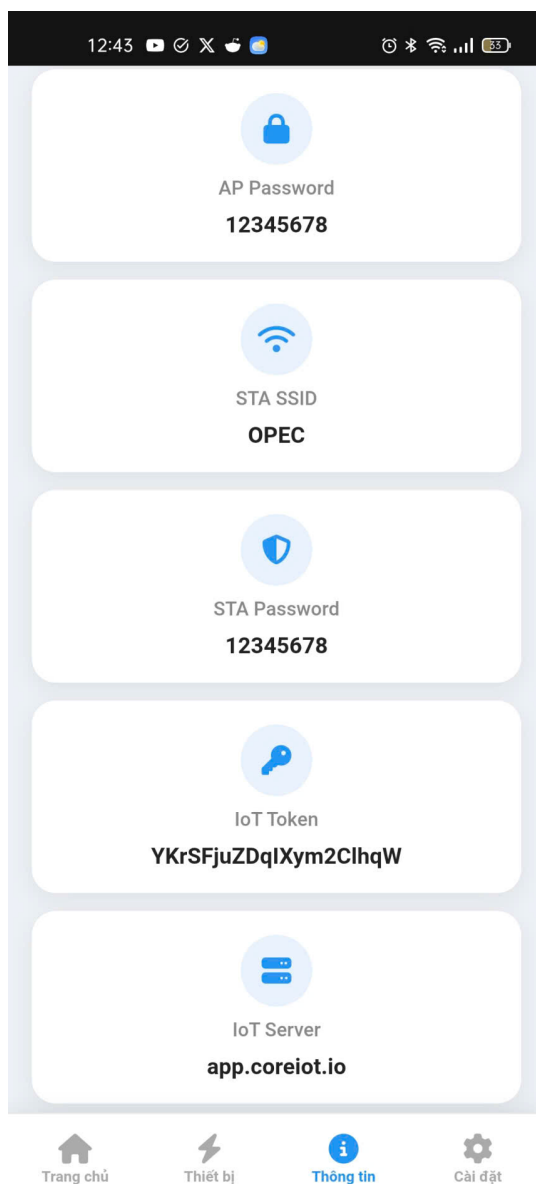
3. **Info:** Địa chỉ IP, uptime, cấu hình AP/STA, thông tin CoreIOT.
4. **Settings:** Form cấu hình WiFi (có nút quét mạng) và CoreIOT token; sau khi lưu thiết bị tự khởi động lại.



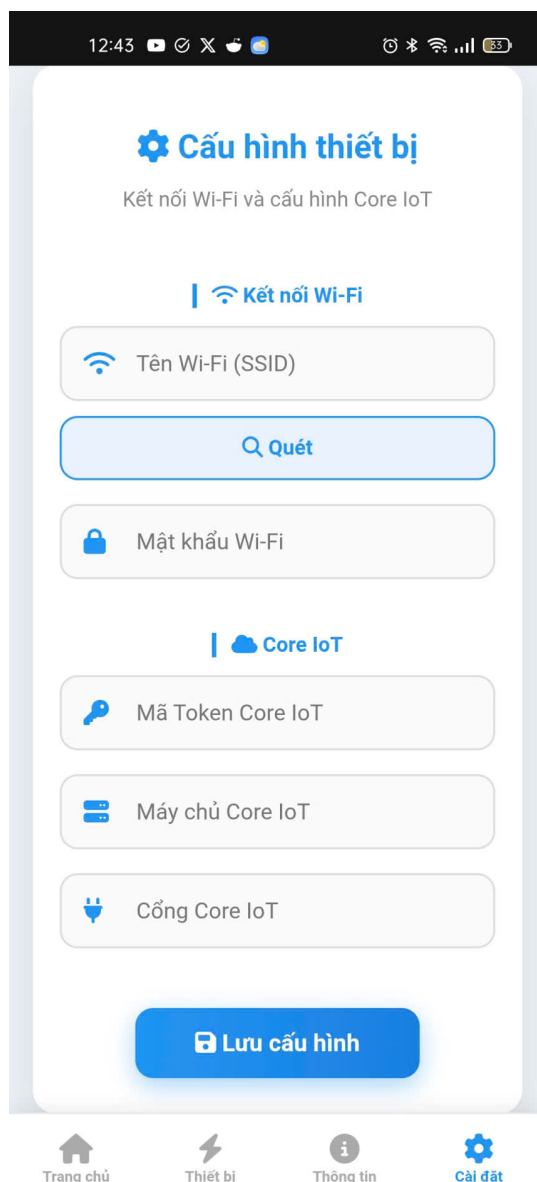
Hình 5.1: Dashboard — Trang Home: gauge nhiệt độ & độ ẩm thời gian thực



Hình 5.2: Dashboard — Trang Device: nút ON/OFF cho LED & NeoPixel và điều khiển relay động qua WebSocket



Hình 5.3: Dashboard — Trang Info: thông tin kết nối WiFi AP/STA



Hình 5.4: Dashboard — Trang Settings: cấu hình WiFi & CoreIOT token

5.5 Task 5: TinyML — Phát Hiện Bất Thường

Yêu cầu: Triển khai mô hình TinyML trên vi điều khiển, mô tả dataset, đánh giá độ chính xác.

5.5.1 Dataset

2000 mẫu tổng hợp (`custom_iot_dataset.csv`):

- **Normal (80%):** $25 \leq T \leq 30C$ và $40 \leq H \leq 60\%$

- **Anomaly (20%):** Nóng ($T > 35C$), Lạnh ($T < 20C$), Ẩm ($H > 70\%$), Khô ($H < 30\%$)

5.5.2 Kiến Trúc Mô Hình ANN

Input(2) $\rightarrow$ Dense(8, ReLU) $\rightarrow$ Dense(1, Sigmoid)

Tổng 33 tham số, kích thước .tflite: **1928 bytes**. Tensor arena: 10 KB.

5.5.3 Tiền Xử Lý Z-score

Hàng số chuẩn hóa nhúng trực tiếp vào firmware:

Listing 5.3: Hàng số Z-score trong tinymml.cpp

```
1 const float SCALER_MEAN[] = {27.4874f, 49.9192f};  
2 const float SCALER_SCALE[] = {4.2157f, 12.0575f};
```

5.5.4 Kết Quả Đánh Giá

- Train accuracy: **99.44%** (epoch 100)
- 6/6 test case biên (nóng, lạnh, ẩm, khô, sát biên) cho kết quả đúng.
- Hạn chế: Dataset tổng hợp, chưa đánh giá trên dữ liệu thực tế từ cảm biến.

5.6 Task 6: CoreIOT Cloud (MQTT)

Yêu cầu: Dẩy sensor data lên CoreIOT, ESP32-S3 ở chế độ STA, dùng đúng access token.

5.6.1 Cấu Hình MQTT

- Thư viện: **PubSubClient**
- Server: `app.coreiot.io`, port 1883
- Topic: `esp/telemetry`
- Xác thực ThingsBoard: username = access token, password = rỗng
- ClientID: `"ESP32_" + WiFi.macAddress()` (đảm bảo tính duy nhất)

5.6.2 Đồng Bộ Với WiFi

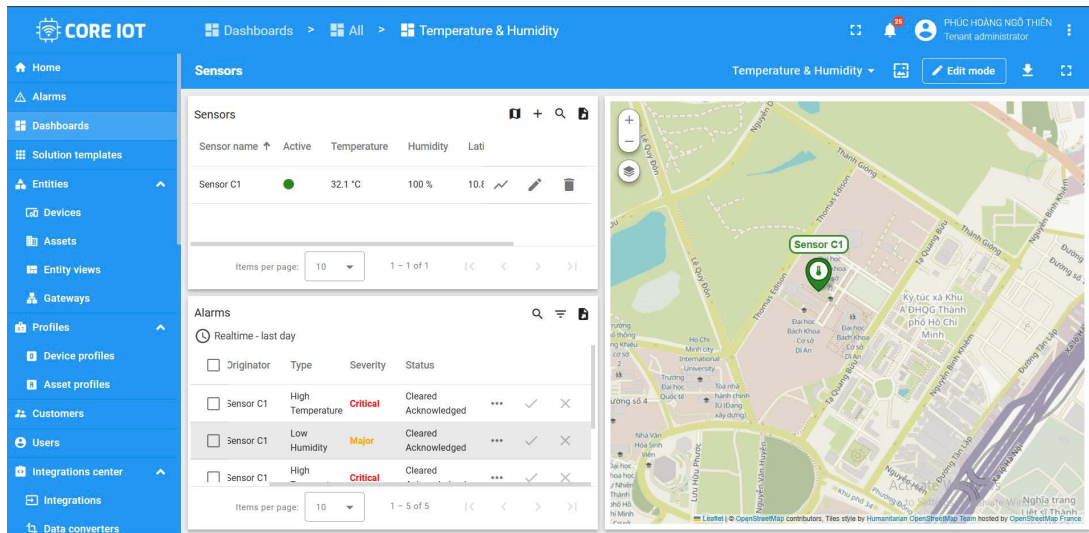
Task `task_core_iot` chờ semaphore `xBinarySemaphoreInternet` — được give trong WiFi event callback `ARDUINO_EVENT_WIFI_STA_GOT_IP` khi STA nhận được IP. Đảm bảo không cố kết nối MQTT trước khi có Internet.

5.6.3 Payload Telemetry

Mỗi 10 giây, task đọc `xSensorQueue` và publish JSON:

Listing 5.4: Cấu trúc payload gửi lên CoreIOT

```
1 {"temperature": 28.5, "humidity": 55.3}
```



Hình 5.5: Dashboard CoreIOT nhận telemetry nhiệt độ và độ ẩm từ ESP32-S3 [5]

6 Cơ Chế Đồng Bộ (Semaphore & Queue)

6.1 Tổng Quan Các Đối Tượng RTOS

Tất cả Queue và Semaphore khai báo trong `include/global.h`, khởi tạo trong `src/global.cpp` hàm `init_globals()`.

Tên	Loại	Depth	Mục đích
xSensorQueue	Queue	1 (Mailbox)	Truyền SensorData từ monitor sang LCD/WebServer/TinyML

Bảng 6.1: Queue trong hệ thống

Semaphore	Give bởi	Take bởi
xTempLowSemaphore	temp_humi_monitor	led_blinky
xTempNormalSemaphore	temp_humi_monitor	led_blinky
xTempHighSemaphore	temp_humi_monitor	led_blinky
xHumiLowSemaphore	temp_humi_monitor	neo_blinky
xHumiNormalSemaphore	temp_humi_monitor	neo_blinky
xHumiHighSemaphore	temp_humi_monitor	neo_blinky
xStatusNormalSemaphore	monitor + tiny_ml	lcd_task
xStatusWarningSemaphore	monitor + tiny_ml	lcd_task
xStatusCriticalSemaphore	temp_humi_monitor	lcd_task
xBinarySemaphoreInternet	WiFi event callback	task_core_iot

Bảng 6.2: Danh sách 9 Binary Semaphore và 1 Internet Semaphore

6.2 Pattern: Edge Detection

Producer chỉ give semaphore khi **phát hiện thay đổi mức** (không give ở mỗi chu kỳ đọc). Consumer dùng `xSemaphoreTake(..., 0)` (non-blocking poll) và duy trì state cuối cùng. Tránh semaphore bị "cộng dồn" và giảm tải FreeRTOS scheduler.

6.3 Pattern: Mailbox Queue với Overwrite

`xSensorQueue` (depth=1) hoạt động như mailbox:

- Producer: `xQueueOverwrite` — ghi đè giá trị cũ, không bao giờ block.
- Consumer: `xQueuePeek` — đọc không xóa, nhiều task đọc cùng lúc an toàn.

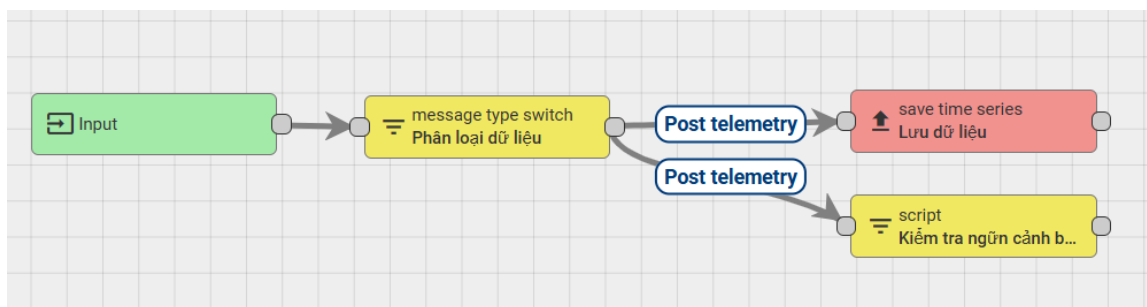
7 ThingsBoard Rule Chain

Rule Chain là tính năng xử lý dữ liệu phía cloud của nền tảng CoreIOT (ThingsBoard), cho phép định nghĩa luồng xử lý message theo dạng đồ thị node — không cần thay đổi firmware ESP32-S3.

7.1 Luồng Dữ Liệu

Mỗi message MQTT từ thiết bị đi qua 4 bước:

1. **Input** — Node *Message Type Switch* tiếp nhận message và định tuyến theo loại (POST_TELEMETRY_REQUEST).
2. **Phân loại dữ liệu** — Node *Script (Transform)* chạy JavaScript kiểm tra giá trị *temperature* và *humidity*, gán nhãn trạng thái **NORMAL** / **WARNING** / **CRITICAL** vào metadata.
3. **Lưu dữ liệu** — Node *Save Timeseries* ghi telemetry vào database ThingsBoard để hiển thị lịch sử trên dashboard.
4. **Kiểm tra logic cảnh báo** — Node *Create & Clear Alarm* tự động tạo hoặc xóa alarm dựa trên ngưỡng T/H, hiển thị cảnh báo trực tiếp trên CoreIOT dashboard.



Hình 7.1: Rule Chain trên CoreIOT: luồng Input → Phân loại → Lưu dữ liệu / Kiểm tra cảnh báo

7.2 Node Script JavaScript

Node Script nhúng trực tiếp logic ngưỡng — đồng nhất với ngưỡng đã định nghĩa trong `include/global.h` của firmware:

```
1 var temperature = msg.temperature;  
2 var humidity = msg.humidity;  
3  
4 // Kiểm tra nếu nhiệt độ ngoài khoảng 25-30 HOẶC độ ẩm dưới 40%  
5 if (temperature < 25 || temperature > 30 || humidity < 40) {  
6     return 'True';  
7 }  
8  
9 return 'False';
```

Hình 7.2: Node Script JavaScript: kiểm tra ngưỡng T/H và gán nhãn trạng thái vào metadata

7.3 Gán Rule Chain Vào Device Profile

Rule Chain được gán vào **Device Profile** của thiết bị ESP32-S3 thay vì Rule Chain mặc định, giúp cách ly logic xử lý theo từng loại thiết bị và dễ tái sử dụng khi thêm thiết bị mới:



Temperature Sensor

Device profile details

<

Details

Transport configuration

Calculated fields

Alarm rules (3)

Device provisioning >

Open details page

Export device profile

Make device profile default

Delete device profile

Copy device profile Id

Name*

Temperature Sensor

Default rule chain

C1

Mobile dashboard

Used by mobile application as a device details dashboard

Queue

Default edge rule chain

Temperature & Humidity Sensors (Remote Facility)

Activate Windows
Go to Settings to activate Windows.

Used on edge as rule chain to process incoming data for devices of this device profile

Hình 7.3: Gán Rule Chain tùy chỉnh vào Device Profile trên CoreIOT

7.4 Lợi Ích

- **Tách biệt edge và cloud:** Firmware chỉ publish JSON gốc; toàn bộ logic phân loại và cảnh báo nằm trên cloud, chỉnh sửa không cần flash lại thiết bị.
- **Alarm tự động:** ThingsBoard tạo/xóa alarm và có thể kích hoạt thông báo (email, Telegram) qua Rule Chain mà không cần code thêm phía ESP32-S3.
- **Lưu trữ có cấu trúc:** Dữ liệu được gán nhãn trạng thái trước khi lưu, dễ truy vấn và vẽ biểu đồ lịch sử theo mức cảnh báo.

8 Mã Nguồn — Các Đoạn Code Quan Trọng

8.1 Khởi Tạo Hệ Thống

Listing 8.1: init_globals() trong src/global.cpp

```
1 void init_globals() {
2     xSensorQueue = xQueueCreate(1, sizeof(struct SensorData))
3     ;
4     xTempLowSemaphore = xSemaphoreCreateBinary();
5     xTempNormalSemaphore = xSemaphoreCreateBinary();
6     xTempHighSemaphore = xSemaphoreCreateBinary();
7     xHumiLowSemaphore = xSemaphoreCreateBinary();
8     xHumiNormalSemaphore = xSemaphoreCreateBinary();
9     xHumiHighSemaphore = xSemaphoreCreateBinary();
10    xStatusNormalSemaphore = xSemaphoreCreateBinary();
11    xStatusWarningSemaphore = xSemaphoreCreateBinary();
12    xStatusCriticalSemaphore = xSemaphoreCreateBinary();
13    xBinarySemaphoreInternet = xSemaphoreCreateBinary();
14    xSemaphoreGive(xStatusNormalSemaphore); // Khởi động o
        Normal
}
```

8.2 Edge Detection Semaphore (Task 1)

Listing 8.2: Producer - edge detection trong temp_humi_monitor.cpp

```
1 if (temperature < TEMP_NORMAL_LOW && last_temp >=
2     TEMP_NORMAL_LOW) {
3     xSemaphoreGive(xTempLowSemaphore);
4 } else if (temperature >= TEMP_NORMAL_LOW && temperature <=
5     TEMP_NORMAL_HIGH
6     && (last_temp < TEMP_NORMAL_LOW || last_temp >
7     TEMP_NORMAL_HIGH)) {
8     xSemaphoreGive(xTempNormalSemaphore);
9 } else if (temperature > TEMP_NORMAL_HIGH && last_temp <=
10    TEMP_NORMAL_HIGH) {
11    xSemaphoreGive(xTempHighSemaphore);
12 }
13 last_temp = temperature;
```


Listing 8.3: Consumer - non-blocking poll trong led_blinky.cpp

```
1 if (xSemaphoreTake(xTempHighSemaphore, 0) == pdTRUE)
    blinkInterval = 200;
2 else if (xSemaphoreTake(xTempNormalSemaphore, 0) == pdTRUE)
    blinkInterval = 1000;
3 else if (xSemaphoreTake(xTempLowSemaphore, 0) == pdTRUE)
    blinkInterval = 2000;
```

8.3 Mailbox Queue (Task 3)

Listing 8.4: Producer dùng xQueueOverwrite - temp_humi_monitor.cpp

```
1 struct SensorData data = {temperature, humidity, lcd_status};
2 xQueueOverwrite(xSensorQueue, &data);
```

Listing 8.5: Consumer dùng xQueuePeek - lcd_task.cpp

```
1 struct SensorData currentData;
2 xQueuePeek(xSensorQueue, &currentData, pdMS_TO_TICKS(100));
```

8.4 TFLite Micro Inference (Task 5)

Listing 8.6: Z-score và inference trong tinymml.cpp

```
1 input->data.f[0] = (sensorData.temperature - SCALER_MEAN[0])
    / SCALER_SCALE[0];
2 input->data.f[1] = (sensorData.humidity - SCALER_MEAN[1])
    / SCALER_SCALE[1];
3 interpreter->Invoke();
4 float score = output->data.f[0];
5 if (score > 0.5f) xSemaphoreGive(xStatusWarningSemaphore);
6 else xSemaphoreGive(xStatusNormalSemaphore);
```

8.5 MQTT CoreIOT (Task 6)

Listing 8.7: Publish telemetry trong task_core_iot.cpp

```
1 xSemaphoreTake(xBinarySemaphoreInternet, pdMS_TO_TICKS(5000))  
    ;  
2 xSemaphoreGive(xBinarySemaphoreInternet); // Tra lai cho  
    task khac  
3  
4 mqttClient.connect(clientId.c_str(), ACCESS_TOKEN, "");  
5  
6 char payload[64];  
7 snprintf(payload, sizeof(payload),  
8     "{ \"temperature\":%.2f, \"humidity\":%.2f }",  
9     data.temperature, data.humidity);  
10 mqttClient.publish("esp/telemetry", payload);
```

9 Kết Luận

9.1 Tóm Tắt Thành Tựu

- **Task 1:** Hoàn thành — LED GPIO 48 chớp đúng 3 tốc độ, semaphore edge detection hoạt động chính xác.
- **Task 2:** Hoàn thành — NeoPixel GPIO 45 hiển thị 3 màu theo độ ẩm, có toggle ON/OFF qua dashboard.
- **Task 3:** Hoàn thành — không biến toàn cục sensor; LCD cập nhật 500 ms/lần với 3 trạng thái qua mailbox queue.
- **Task 4:** Hoàn thành — dashboard 4 trang, gauge thời gian thực, điều khiển LED/NeoPixel và relay động; *tồn đọng*: relay chưa lưu LittleFS sau reboot.
- **Task 5:** Đã triển khai — TinyML ANN 1928 bytes, 99.44% accuracy trên dataset tổng hợp^[4], 6/6 test case biên đúng; chưa đánh giá trên dữ liệu thực.
- **Task 6:** Code hoàn chỉnh — MQTT kết nối, xác thực, publish JSON, reconnect; cần cấu hình access token thực qua trang Settings.
- **Rule Chain:** Triển khai thành công trên CoreIOT — phân loại dữ liệu, lưu timeseries và tạo alarm tự động theo ngưỡng T/H.

```
[DHT20] T=33.34 H=100.00 stat: critical  
[CoreIoT] Publishing → esp/telemetry : {"temperature":33.33873749,"humidity":99.99990845}  
[CoreIoT] Result: OK  
[TinyML] Input: 33.3,100.0 | Score: 0.9998 | pred: abnormal
```

Hình 9.1: Serial Monitor hiển thị kết quả inference TinyML trên ESP32-S3

9.2 Bài Học Kinh Nghiệm

- **Mailbox pattern** (`xQueueCreate(1) + xQueueOverwrite + xQueuePeek`): tối ưu cho dữ liệu "chỉ cần giá trị mới nhất", không block producer, không cạnh tranh giữa các consumer.
- **Edge detection semaphore**: tốt hơn level detection vì tránh tràn binary semaphore và giảm tải scheduler.
- **WiFi AP+STA đồng thời**: giải quyết mâu thuẫn giữa Task 4 (cần AP) và Task 6 (cần STA).
- **TFLite Micro**: yêu cầu định dạng `.tflite flatbuffer` từ TensorFlow/Keras; kích thước tensor arena cần điều chỉnh theo kiến trúc model.
- **Init trước task creation**: `init_globals()` phải gọi trong `setup()` trước `xTaskCreate()` để tránh race condition.

Mã Nguồn

Toàn bộ mã nguồn của dự án được lưu trữ công khai trên GitHub:

<https://github.com/thienphucope/iot252>

Repository bao gồm mã nguồn PlatformIO (C++), notebook huấn luyện TinyML (Python/Jupyter), dataset tổng hợp và tài liệu kỹ thuật.

Tài Liệu Tham Khảo

Tài liệu

- [1] Real Time Engineers Ltd., *FreeRTOS — Real-time operating system for microcontrollers*, <https://www.freertos.org>, truy cập 2026.
- [2] Espressif Systems, *ESP32-S3 Technical Reference Manual*, Espressif Systems, 2022. https://www.espressif.com/sites/default/files/documentation/esp32-s3_technical_reference_manual_en.pdf
- [3] TensorFlow Authors, *TensorFlow Lite for Microcontrollers*, <https://www.tensorflow.org/lite/microcontrollers>, truy cập 2026.
- [4] S. (smmmmmmmmmmmmmmm), *Anomaly Detection in IoT Devices — Synthetic Dataset (2000 samples)*, Kaggle, 2024. <https://www.kaggle.com/datasets/smmmmmmmmmmmmmmmm/anomaly-detection-in-iot-devices>
- [5] CoreIOT Platform (ThingsBoard-based), *CoreIOT Cloud Dashboard*, <https://app.coreiot.io>, truy cập 2026.
- [6] M. Hightower et al., *ESPAsyncWebServer — Async HTTP and WebSocket Server for ESP32*, GitHub, 2021. <https://github.com/me-no-dev/ESPAsyncWebServer>
- [7] N. O’Leary, *PubSubClient — MQTT Client for Arduino*, <https://pubsubclient.knolleary.net>, truy cập 2026.
- [8] Adafruit Industries, *Adafruit NeoPixel Library for Arduino*, GitHub, 2023. https://github.com/adafruit/Adafruit_NeoPixel
- [9] PlatformIO Labs, *PlatformIO — Professional collaborative platform for embedded development*, <https://platformio.org>, truy cập 2026.
- [10] Nhóm sinh viên, *Mã nguồn dự án: YoloUNO PlatformIO-RTOS*, GitHub, 2026. <https://github.com/thienphucope/iot252>